A Smaller Complete iPRA Checker and an Obstruction to Positive Axiomatizations

Explicit rules, both interpretation composites, a pinned source audit, and a quantified symmetry theorem

Research prepared for Vladimir Reshetnikov

10 November 2026

Abstract

We give a complete proof presentation of full first-order intuitionistic primitive recursive arithmetic and an independently executable checker with 612 normalized Python bytes, 266 significant tokens, and 310 syntax-tree nodes. Maximum-compression gzip occupies 360 bytes and is reported only as information. The current supplied corpus has separate minima of 616 bytes, 268 tokens, and 300 nodes. Thus the selected checker improves source size and tokens, but not the syntax-tree minimum. Its ordinary list check does not rely on divergent malformed-string processing. The construction combines the existing arity representation with the existing predecessor computation interface; it is not a new numerical or logical axiom basis.

The new mathematical contribution is a broader obstruction to positive axiomatizations. Eventually constant powers have exact coordinatewise truth for formulas built from atoms using conjunction and both quantifiers, with arbitrary finite quantifier alternation. They satisfy induction for all these formulas, all standard-true implications between them, and the corresponding negative constraints. Nevertheless, after any finite set of parameters is named, they retain a nontrivial involution. This excludes ordinary first-order bi-interpretability with iPRA, even using additional sorts, arbitrary formula-defined domains and graphs, finite tuples, and definable quotient equality. The result extends an obstruction in the supplied drafts and is not a source-size lower bound.

The archive supplies the complete primitive rule inventory, attributed arithmetic and logical bridge proofs, executable proof certificates, negative controls, an exhaustive source census, and reproducible measurements. The broader request for a significant new simplest axiomatization remains unresolved. No absolute minimality or proof-assistant verification is claimed.

Precise outcome. A smaller complete implementation and a stronger non-bi-interpretability theorem are established. The positive arithmetic foundation is inherited. A four-byte implementation gain should not be mistaken for the requested foundational breakthrough.

Contents

1 The result, the comparison, and the unresolved requirement **3**

2 A complete native presentation of full intuitionistic PRA **4**

3 A composed arity and predecessor checker **9**

4 Conjunctive induction still leaves a symmetry obstruction **14**

5 Executable evidence, attribution, and research limits **18**

Attribution and the complete inherited bridge **21**

A Complementary introductions with one local assumption **21**

B Arithmetic strength and both ordinary interpretation composites **26**

C Internal scalar predecessor bootstrap **28**

D From literal-equation induction to full intuitionistic PRA **32**

E Pairing, primitive recursion, and internal numerical composites **38**

Reference proof for the original arity codec **40**

F A uniform arity encoding and its checking theorem **40**

1 The result, the comparison, and the unresolved requirement

The intended target is full first-order *intuitionistic* primitive recursive arithmetic, not just its equational fragment or classical PRA. The familiar numerical primitives and quantifier-free induction can be presented with either classical or intuitionistic first-order logic; that distinction matters for a claim of bi-interpretability [13, 14]. We fix the intuitionistic theory throughout. The full logical interface will be derived from an explicitly listed native calculus rather than left as an unmeasured background theorem prover.

1.1 A pinned and exhaustive implementation baseline

The comparison uses commit

`fb2dbd2a33c6b132671a54b029d0bf3cd98c633d`.

There are thirteen ZIP archives directly in `docs/drafts` at this revision. Recursively including embedded historical archives gives 28 distinct archive byte strings and 370 distinct Python source byte strings. The inventory is SHA-256 deduplicated and keeps every original archive and member path. Small build scripts, test runners, and producer programs are not complete proof checkers. Complete historical controls inside `reference/` or `ancillary/` do count.

Every source at or below at least one final coordinate minimum was individually classified. There are 31 such sources: three complete checkers and 28 supporting programs. The current minimum byte count is *616*, not the older 672 quoted in several supplied reports. The current minimum token and AST counts are 268 and 300. The complete census and classifications are in `evidence/`.

Complete standalone program	Bytes	Tokens	AST	Gzip 9
Current byte leader	616	269	311	366
Current AST leader	649	268	300	367
Current ordinary-list predecessor checker	629	269	307	373
Current matched old-flat control	628	269	306	374
Selected arity/predecessor composition	**612**	**266**	310	360
Ancillary shorter-byte composition	597	267	315	356
Ancillary smaller-AST composition	639	267	298	360

Bytes means the size after removing genuine docstrings and comments, applying `ast.unparse`, and adding one final newline. Significant tokens exclude layout and comment tokens. AST counts include every node of the parsed module. Gzip is measured on the exact delivered raw source with `gzip.compress(data, compresslevel=9, mtime=0)`; the complete gzip member, including header and trailer, is counted. Compression is not an optimization objective. The runtime, source hashes, raw sizes, and normalized-source gzip sizes are recorded with the measurements.

These are whole-program measurements. They include the JSON input, traversal and scope checks, arithmetic supply, logical rules, recursive proof checking, and success output. The Python runtime and standard JSON library are common infrastructure. No producer, theorem library, arithmetical evaluator, or precomputed rule table is imported by the selected program. Its separately measured readable expansion is also executable.

The selected program improves the current byte coordinate by $616 - 612 = 4$ bytes, approximately 0.65%, and the token coordinate by two tokens. It has ten more AST nodes than the separate AST minimum. The two ancillary programs show additional tradeoffs; they do not make the selected

program a simultaneous record in all coordinates. The smaller-AST opener inherits a qualification concerning malformed strings and CPython's recursion limit, so the ordinary-list program is the main artifact.

1.2 What is new and what is inherited

The arithmetic basis $0, S, I$ and the native Eq/Q/D inference rules are inherited. The arity codec is inherited from [3], the predecessor packet from [2], and the exit-status rejection convention already appears in [2]. We prove that these mechanisms compose, including an exact simulation of arithmetic packets and a direct justification of zero-witness introduction. This supplies a useful smaller complete checkpoint, but does not establish a new foundational axiom basis.

The mathematical extension is the theorem of Section 4. The earlier eventually constant model argument covers equations and finite conjunctions. Exact transfer in the same model persists through both quantifiers, even with arbitrary finite alternation. Consequently even a much richer class of positive axioms and induction schemas leaves an obstruction to bi-interpretability. The interpretation may use a different domain, finite tuples, multiple sorts, arbitrary definable graphs, and quotient equality. The argument does not rely on term-definability of witnesses.

1.3 Why the word "lower bound" needs qualification

A minimum among the submitted implementations is an observed envelope. It is an upper bound on the unknown optimum, not a lower-bound theorem for every possible axiomatization. Source size also depends on a fixed programming language, runtime interface, normalization convention, and certificate encoding. We preserve the corpus's conventions so that the comparison is reproducible, and do not infer absolute simplicity from it.

The requested significant new simplest axiomatization has not been established. The numerical and logical presentation below is exact and bi-interpretable with iPRA, but its basis is already present in the drafts. The size improvement and new obstruction are stated separately so that neither is used to overstate the other.

2 A complete native presentation of full intuitionistic PRA

This section specifies the mathematical theory checked by the program. It is the inherited Choice-DI scalar-iteration presentation [2, 3]. Its rules, numerical operations, and interpretation proofs are retained explicitly. The new executable is a representation of this theory, rather than a different arithmetical theory whose equivalence is only conjectural.

2.1 Numerical syntax and the two computation schemas

There is one numerical sort, with finite terms

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only the update body h binds u. The counter n and initializer a do not bind it. Other variables in h are passive numerical parameters and remain fixed during iteration. In every retained context, the

following are axiom schemas:

$$I(0, a; u.h) = a, \qquad \text{(I0)} \tag{1}$$
$$I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \qquad \text{(IS)} \tag{2}$$

The update sees only its previous value, not the old counter.

The binder is a finite-description notation with an ordinary first-order elaboration. Beginning with $0, S$, each earlier-stage term $h(u, \vec{p})$ admits a new symbol $i_h(n, a, \vec{p})$ with the two displayed defining equations. Each expression and derivation involves only finitely many stages. This introduces neither a function sort nor an object variable ranging over programs. In particular, it does not postulate a total universal evaluator for all primitive-recursive descriptions.

2.2 Formula syntax and all primitive logical rules

Formulas are finite expressions

$$A ::= (s = t) \mid \mathsf{Q}_y A \mid \mathsf{D}_{x,y} A.$$

The variables x, y bound by D are distinct. Contexts Γ are finite reusable lists of formulas. Substitution always avoids capture; local bound variables are renamed as needed. The designated contradiction is the equation $0 = S0$. The intended readings, which will be proved exact rather than assumed as extra inference rules, are

$$\mathsf{Q}_y A(y) \equiv A(0) \to \forall y A(y), \qquad \mathsf{D}_{x,y} A(x, y) \equiv \exists x \big(A(x, 0) \to \forall y A(x, y)\big).$$

Together with I0 and IS, the following list is complete:

$$\frac{}{\Gamma \vdash A}\ (A \in \Gamma) \quad \text{(Hyp)},$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \quad \text{(Tr)},$$

$$\frac{\Gamma \vdash \mathsf{Q}_y A(y) \qquad \Gamma \vdash A(0)}{\Gamma \vdash A(t)} \quad \text{(QE)},$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} A(x, y) \qquad \Gamma, \mathsf{Q}_y A(b, y) \vdash F}{\Gamma \vdash F} \quad \text{(DE)},$$

$$\frac{\Gamma, A(t, 0) \vdash A(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} A(x, y)} \quad \text{(DI-case)},$$

$$\frac{\Gamma, E(t, z) \vdash E(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} E(x, y)} \quad \text{(DI-progress)}.$$

In Tr, $F(v)$ is any fixed native formula template, including a compound one. In DE, b is fresh for Γ, F and the fixed parameters of the matrix. In both introductions, z is fresh for Γ, the chosen witness t, and the fixed matrix parameters. The displayed local assumptions are discharged. In DI-progress, $E(x, y)$ must be a *literal equation*. An equation inside a quantified formula does not qualify. DI-case is available for both equations and compound formulas.

There are no additional primitive equality, successor, induction, or ordinary logical axioms or rules. In particular, unrestricted progress on compound formulas is not part of the theory. The program checks the root-constructor restriction directly through its arity representation.

2.3 First derived principles

Weakening, exchange, contraction, renaming, numerical substitution, and ordinary-assumption cut are admissible by induction on finite derivations. Rename eigenparameters before inserting a proof or term. Numerical substitution leaves an induction matrix's root constructor unchanged, so the equation-only restriction survives these transformations.

Reflexivity is derived. Let $B(p) = I(0, p; v.0)$, so I0 proves $B(p) = p$. Use that equation as both premises of Tr with template $F(w) := (w = p)$. The result is $p = p$. Transport then supplies symmetry, transitivity, and congruence for every fixed numerical expression, as well as transport inside arbitrary native formulas.

The auxiliary closure $\mathsf{Q}_y A$ is interderivable with $\mathsf{D}_{x,y} A$ when x is unused. From $\mathsf{Q}_y A$, DI-case specializes the closure under its base assumption. Conversely, DE opens the paired formula to exactly $\mathsf{Q}_y A$, and Hyp supplies the continuation. Thus the public use of Q introduces no independent logical strength.

With an unused existential coordinate, DI-progress, DE, and QE derive the contextual equation-induction rule

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}, \qquad z \text{ fresh.} \tag{EqInd}$$

The construction first proves $\mathsf{D}_{x,y} E(y)$ by progress, opens its closure by DE, and applies QE to the separately proved base. The same unused-coordinate construction with DI-case gives numerical zero/successor cases for an arbitrary formula:

$$\frac{\Gamma \vdash A(0) \qquad \Gamma \vdash A(Sz)}{\Gamma \vdash A(t)}, \qquad z \text{ fresh.} \tag{Cases}$$

Only EqInd permits the current instance as an induction assumption. These two rules must not be conflated.

2.4 Recovering ordinary intuitionistic logic

Set $1 = S0$, $2 = S1$, and define a numerical selector

$$C(v, a, b) = I(v, a; u.b).$$

The computation schemas give $C(0, a, b) = a$ and $C(Sv, a, b) = b$. These two endpoint identities are enough to construct a formula gate by external recursion on each fixed formula:

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y} A) := \mathsf{D}_{x,y} G_v A.$$

First replace an auxiliary Q by its unused-witness D version. Then $G_0 A$ is provable and $G_{Sv} A$ is interderivable with A. Padding by $\mathsf{D}_{u,y} G_y A$ preserves a formula and permits binder depths to be aligned.

For equations put

$$M_v(s = t, r = w) := (C(v, s, r) = C(v, t, w)).$$

For paired formulas align binders and recurse on their bodies. If just one input is an equation, pad it first. The maximum remaining binder depth decreases, so this is a finite external construction. Derived binder congruence gives

$$M_0(A, B) \dashv\vdash A, \qquad M_{Sv}(A, B) \dashv\vdash B.$$

Let $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the finite computations $p(1) = 1$ and $p(2) = 0$ are needed for the following logical macros, with all unused coordinates fresh:

$$\begin{aligned} A \to_* B &:= \mathsf{D}_{u,z} M_z(A, B), \\ A \wedge_* B &:= \mathsf{D}_{u,z} G_z M_{p(z)}(A, B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &:= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z), \\ \bot_* &:= (0 = 1). \end{aligned}$$

For implication introduction, the retained zero instance is the antecedent and the successor instance is the consequent. For elimination, open the closure and specialize at 1. Conjunction introduction uses the gate's provable zero case; its projections specialize the positive gate at $1, 2$. Existential introduction uses the proposed numerical witness; elimination opens a fresh witness, proves the gate's zero instance, and specializes at 1. Disjunction uses numerical cases on the selected witness. Universal elimination uses its explicit zero conjunct or the successor closure; universal introduction uses a fresh arbitrary parameter. Every ordinary introduction and elimination rule is proved in the first appendix, including its eigenvariable conditions.

Numerical cases now prove

$$x = 0 \ \vee_* \ \exists_* y\, (x = Sy).$$

When a target occurs in a retained context, one uses this equality-based case split and Tr; one does not substitute into the retained context without justification. Under $0 = 1$, transporting the selector endpoint identities yields any numerical equation. External induction on formula syntax using DI-case gives explosion for every native formula. This recovers genuine intuitionistic falsity without adding excluded middle for arbitrary formulas.

2.5 Recovering all primitive-recursive arithmetic

There is a substantial inherited arithmetic argument behind the small primitive basis. The appendices give its finite terms and proofs, in the following noncircular order:

1. Scalar iteration, cases, and literal-equation induction construct predecessor P, with $P(0) = 0$ and $P(Sx) = x$.
2. A primitive-recursive characteristic of *actual equality* is proved correct on the entire domain, not merely on standard numerals. Boolean characteristics of fixed quantifier-free formulas then reduce their induction to EqInd.
3. Quantifier-free induction proves the required pairing and projection identities. Pairing was not presupposed in the construction of predecessor.
4. Pairing compiles counter-aware primitive recursion into scalar iteration with a passive initializer.

Here is the last compiler explicitly. Suppose a primitive-recursive description has base $g(\vec{p})$ and step $h(n, v, \vec{p})$. Let π, π_0, π_1 be the proved pairing and projections, and assume the smaller descriptions have already been translated. Define

$$\begin{aligned} V(n, \vec{p}) &= I\big(n, \pi(0, g^*(\vec{p})); \ u.\pi(S\pi_0(u), h^*(\pi_0(u), \pi_1(u), \vec{p}))\big), \\ f^*(n, \vec{p}) &= \pi_1(V(n, \vec{p})). \end{aligned}$$

Equation induction proves $\pi_0(V(n,\vec{p})) = n$. The projection identities then prove the required base and step equations for f^*. Composition and projections translate directly. The compilation is an external recursion on a finite function description; it is not a single object-language evaluator for all primitive-recursive programs.

Successor injectivity follows by applying predecessor. If $Sx = 0$, the selector distinguishes the two sides and derives $0 = 1$; hence successor separation follows under the decoded falsity. Together with coverage, full intuitionistic logic, the PR compiler, and quantifier-free induction, this recovers the usual first-order axioms of iPRA.

2.6 Both interpretations and both composites

For a completely ordinary first-order comparison theory, let T_I have the staged iterator symbols, intuitionistic first-order equality logic, I0, IS, EqInd, and

$$x = 0 \vee \exists y(x = Sy), \qquad (0 = S0) \to \bot.$$

The last bridge is explicit. Positive computation equations alone have a singleton ordinary model; naming $0 = S0$ as native falsity is not a reason to omit the bridge from T_I.

Theorem 2.1 (Full-domain bi-interpretability)**.** *The native calculus is a faithful presentation of T_I. It is bi-interpretable and equiconsistent with full first-order iPRA. The numerical domains are unchanged and equality is actual equality. Both domain composites are internally isomorphic to the identity by the relation $x = y$. Both formula composites are derivably equivalent to the original formulas.*

Proof. Interpret a fixed scalar iterator in iPRA by primitive recursion whose step ignores the old counter. Preserve numerical equality and use

$$(\mathsf{Q}_y A)^\circ = A^\circ(0) \to \forall y A^\circ(y), \qquad (\mathsf{D}_{x,y} A)^\circ = \exists x\big(A^\circ(x,0) \to \forall y A^\circ(x,y)\big).$$

Tr, QE, and DE are valid under these readings. DI-case is valid by successor coverage; DI-progress is valid by equation induction, under the temporarily assumed zero instance. I0 and IS are the primitive recursion equations. Thus every native proof translates to an iPRA proof.

In the reverse direction, translate every PR function description by the scalar compiler and every ordinary connective and quantifier by the displayed macros. The derived rules and arithmetic bridge validate each logical and nonlogical axiom and each inference of iPRA.

For numerical round trips, two scalar iterations with the same seed and provably equal fixed updates are equal at every counter by EqInd. The analogous uniqueness proof applies to each compiled primitive recursion. External induction on a finite function description therefore identifies its double translation with the original function, including its passive parameters. This also proves coherence of the staged binder notation. The domain is the whole numerical domain in each direction; $x = y$ is consequently an internally total, bijective isomorphism of the composite with the original structure.

For the nontrivial native formula composite, prove

$$\mathsf{D}_{x,y} A \dashv\vdash \exists_* x\big(A(x,0) \to_* \forall_* y A(x,y)\big).$$

Forward, use DE at a fresh witness, assume the base, and specialize the exposed closure at an arbitrary numerical argument. Reverse, eliminate the translated existential, use its implication and universal instance at Sz under the base assumption, and discharge DI-case. The closure composite is the unused-witness instance. External structural induction on formulas, using the derived ordinary

rules, proves both formula composites. All these derivations are internal to the respective theories; standard-model agreement alone is not the argument.

The falsity bridge preserves contradictions in both directions. The effective translations of finite derivations therefore establish equiconsistency. They do not assert an internal proof of either theory's own consistency. □

The proof is a written mathematical reduction with its inherited arithmetic details reproduced below. The executable examples verify particular proof certificates; they are not a Lean or Rocq verification of this metatheorem.

3 A composed arity and predecessor checker

The construction in this section is deliberately a composition of two existing mechanisms in the inspected drafts. It changes the complete certificate checker, not the underlying arithmetic foundation. The main program uses the strict list test `isinstance(a,list)` and has 612 normalized bytes, 266 significant Python tokens, and 310 AST nodes. It is smaller than the current complete-program byte and token minima, 616 and 268, respectively. It does not improve the separate 300-node AST minimum. Its actual source has 612 bytes and its ordinary gzip level-9 encoding has 360 bytes, including the wrapper. These figures use CPython 3.12.14 and zlib 1.3.2.

The inherited arity checker [3] supplies the sorted tree representation and the logical branches. The uniform-predecessor checker [2] supplies the arithmetic packet interface. Both have ordinary exit-status rejection in this composition. The underlying native calculus, numerical bootstrap, reconstruction of ordinary intuitionistic logic, and whole-domain bi-interpretation are the attributed inherited results. Nothing here establishes an absolute minimum or a significant new arithmetic foundation.

3.1 The exact decoded language

Numerical terms are variables, zero, successor, and initialized scalar iteration

$$I(0, a; v.h) = a, \qquad I(Sn, a; v.h) = h[I(n, a; v.h)/v].$$

The native formulas are equations and the paired binder; the checker also uses conditional closures:

$$Q_y B \equiv B(0) \to \forall y\, B(y), \qquad D_{x,y} B \equiv \exists x\, Q_y B(x, y).$$

The displayed equivalences give their ordinary realizations. Their primitive native rules are the inherited case/progress introductions, closure use, paired elimination, equality transport, and computations. Progress remains restricted to literal equations.

Canonical formulas are encoded by

$$s = t \longmapsto [s, t], \qquad D_{x,y} B \longmapsto [B], \qquad Q_y B \longmapsto [B, B, B].$$

Only the third field of an arbitrary ternary formula has mathematical content. The repeated fields in a canonical closure are deliberate redundancy. Multiplication of a unary major array by three produces its closure; multiplication of a binary or ternary major array by three produces a high-arity formula, which denotes the fixed truth $0 = 0$.

The decoder is sorted. A numerical root has protected-coordinate depth three, whereas a formula root has depth zero. Under r protected numerical slots the corresponding depths are $3 + 2r$ and

$2r$. At numerical root, coordinates $0, 1, 2$ are zero aliases; coordinates $3 + 2j, 4 + 2j$ are the two aliases of protected slot j. Negative coordinates admitted by the current floor denote rigid parameters. Canonical inputs use integer coordinates.

A unary numerical node is successor. Its child receives two dummy zero slots. A ternary numerical node $[n, a, h]$ is iteration. Its counter and initializer receive one dummy zero slot; its update receives the actual state slot. Empty numerical arrays, binary numerical annotations, and numerical arrays of arity at least four all denote zero. At formula positions empty arrays, arrays of arity at least four, and admitted scalar padding denote $0 = 0$. These are fixed aliases, not extra predicate or function symbols.

The opener increments depth by

$$5 - \mathrm{len}(a)$$

for every array. The meaningful increments are 4 for paired binding or successor, 3 for equality, and 2 for closure binding or iteration. This is exactly the inherited arity codec. At a meaningful scalar position it preserves an admitted coordinate when $k \le a < d$; otherwise, at odd depth it inserts its image after checking that image at numerical depth three. At even depth scalar padding becomes truth. A missing image at a used numerical hole rejects.

The precise opening invariant. At a meaningful sorted root, a successful opener call implements capture-free filling of the distinguished numerical hole under the decoder just given. The proof is structural induction on the raw tree. For arities one, two, and three, the depth increment is exactly the number of new protected slots plus any required sort change. At ignored annotations and surplus-arity nodes the interpretation is constant, so traversal below them cannot introduce a meaningful free variable. The inserted image is checked at its own numerical root; its free parameters are rigid negative labels, so inserting it below protected slots captures none of them.

This is a conditional semantic invariant. It is not a claim that opening is idempotent on every raw tree, nor that every semantic alias must accept the same certificate. For example, opening a high-arity truth array can alter its ignored children; a later traversal may reject those children without contradicting its constant decoded meaning. Canonical object expressions avoid surplus nodes, while administrative high-arity truth exports are justified separately.

3.2 An exact reduction of the arithmetic branch

A new arithmetic instruction is

$$[\texttt{null}, p, a, h].$$

If the raw predecessor p is truthy, the checker constructs

$$H = [\,[[p], a, h],\ i(h, [p, a, h], 3)\,]. \tag{C1}$$

If p is falsy, it constructs

$$H = [[p, a, h], a]. \tag{C0}$$

It then allocates its next parameter and opens this local hypothesis with image zero. The test selecting the arithmetic branch is the literal null test. Literal zero therefore remains available as a paired-introduction witness.

There is a particularly direct soundness reduction, which also checks the change of numerical opening depth. The old arity checker accepts constructed-computation packets of the form $[m, s, u, t]$, where $s = [n, b, r]$ and falsy t selects computation. Use the following old packets for the new packet

above:

Condition	Corresponding old arity packet
p truthy	$[\,[\,],\,[p,a,h],\,1,\,0\,]$
p falsy	$[\,[\,],\,[0,a,h],\,p,\,0\,]$.

(Packet correspondence)

The empty-array marker is essential in this comparison. In the common continuation, both new marker null and old marker [] make `len(goal) % 2 == marker` false. Both therefore select image zero for the supplied hypothesis, independently of the goal's arity. One must not compare with the old numeric-zero marker and silently assume its image choice is unchanged.

For truthy p, the old destructuring is $n = p, b = a, r = h, u = 1$. Its constructed source and recursive-image opening are exactly (C1), including the opening at depth three. For falsy p, Python's expression u and $[n]$ returns $u = p$, so the old source is exactly (C0), retaining even the same raw zero alias. The unused first field of the old triple may be zero. Thus the generated raw equation, old-floor image checks, next floor, opened goal, opened hypothesis, and continuation context are identical in the two executions. No appeal to a different meaning of a raw numerical coordinate is needed.

Consequently the inherited arithmetic-supply proof applies without alteration. In the state-dependent successor case, the call $i(h, [p, a, h], 3)$ checks its recursive image at numerical root depth three whenever the state is used in a relevant position. This checks the predecessor, initializer, and update at the old floor. Coordinates three and four are the state aliases. An escaping relevant coordinate cannot be hidden by deferring that image check. If the update does not use the state, the successor equation has its ordinary constant-update form and does not require the recursive image. The source itself is still traversed when the supplied equation is opened.

In the base case, every successfully admitted falsy numerical counter in the strict-list profile denotes zero: the relevant aliases are zero, false, numerically equal zero values, and the empty array. Null, strings, and dictionaries reject when used in a meaningful numerical position. This gives an ordinary zero-computation instance.

Fresh parameters and the zero-instance argument. The exact reduction above is useful because it prevents a stronger arithmetic axiom from entering through the proof interface. The inherited supply argument permits an equation $H(\vec{z})$ when its simultaneous zero instance is a computation theorem and $\vec{z}$ is fresh for the retained context and goal. If

$$\Gamma, H(\vec{z}) \vdash F \quad \text{and} \quad \vdash H(\vec{0}),$$

substituting zero throughout the first derivation leaves Γ, F unchanged; cutting in the second derivation gives $\Gamma \vdash F$. This is a finite native proof transformation, not a consistency or reflection assumption.

When the floor drops from K to $K-1$, every relevant new label in $[K-1, K)$ is covered by this simultaneous substitution. For canonical integer syntax only $K-1$ is new. If fractional JSON numbers are accepted as raw aliases, they are additional distinct fresh labels; they must not silently be identified with the designated integer parameter. The known-zero-instance argument covers the finitely many that occur. Previously checked goals and retained hypotheses contain none of these names in meaningful positions. The composition does not need to assert that every raw supplied equation is uniformly a theorem in every newly admitted label.

3.3 The logical branches and zero witnesses

Every truthy-major branch is inherited unchanged from the strict arity checker. The major is proved first. An exact equation endpoint permits transport, while an exact triplicate closure permits

specialization only when the old endpoint is numerical zero. The explicit `not source` condition is retained. The continuation either exports the closure of a paired major at a fresh parameter, or exports a formula with the fixed truth interpretation.

Every remaining falsy non-null marker selects paired introduction. The expression `i(*goal, witness, 2)` enforces the unary shape of a canonical paired formula. Binary equation goals and ternary closure goals supply too many arguments and reject. An empty goal has the constant-truth interpretation, so any accepted administrative empty-goal case is harmless. Surplus-arity goals also fail this call.

For a canonical goal $D_{x,y}B$, opening at depth two preserves y and fills its witness coordinate with the supplied term t. The checker then proves the successor instance at a fresh z. Marker [] provides $B(t, 0)$ as the local assumption. Marker 0 provides $B(t, z)$ only when the opened matrix is a binary, literal equation; for a compound matrix it provides the zero instance instead. These are precisely the inherited case and equation-progress rules.

This argument allows the actual numerical witness zero. It does not identify zero with a missing witness. A missing image is rejected when the witness coordinate is semantically used. If the coordinate is unused, zero is a legitimate mathematical representative. The canonical completeness argument needs only ordinary arrays and integer coordinates, so the additional harmless raw aliases do not increase its logical requirements.

3.4 Completeness, including the zero predecessor

Canonical encoding of the native logical rules follows the inherited arity proof. The only change needed in arithmetic is to handle the syntactic falsity of a literal-zero predecessor in a successor computation. Define the finite term

$$B(p) = I(0, p; w.0).$$

The base computation proves $B(p) = p$, and the syntax of $B(p)$ is a nonempty array. For any update h with state variable v, put

$$F(z) \ :\equiv \ I(Sz, a; v.h) = h[I(z, a; v.h)/v].$$

The new arithmetic branch directly proves $F(B(p))$ because the predecessor $B(p)$ has truthy syntax. A second arithmetic instruction proves $B(p) = p$. Equality transport through the full formula template $F(z)$ then proves $F(p)$. This is an explicit three-node derivation: one boxed successor computation, one zero computation, and one transport. It works in particular for $p = 0$, and works for every well-scoped predecessor if a uniform compiler is preferred.

The zero computation itself is direct: use predecessor zero. All other truthy canonical predecessors may use direct successor packets. The inherited eigenparameter renaming and continuation serialization then compile every finite native proof. The supplied untrusted producer implements this box expansion, combines the exact arity numerical/formula codecs with the uniform-predecessor emitter, and retains the named-context discipline. A checker never imports that producer or any arithmetic evaluator.

It follows that the canonical consequence relation is exactly the inherited native calculus. Its ordinary realization is therefore whole-domain bi-interpretable and equiconsistent with iPRA by the already supplied logical and arithmetic bridge. The interpretations and their identity numerical round trips are unchanged. This transfer is a consequence of equality of the presented calculus, not an inference from finite testing or from agreement on standard numerals.

3.5 Ancillary profiles and the exact validation boundary

The byte profile composes the inherited exact-JSON list predicate and short-circuit endpoint expression. On canonical object formulas the replacement endpoint is a nonempty array, so this expression has the same behavior as the explicit conditional. On a raw falsy truth alias it can discard the conversion and retain the old goal. It then requires a proof of that old goal and does not introduce an unsound implication. The profile measures 597 bytes, 267 tokens, and 315 AST nodes; its actual gzip-9 stream has 356 bytes.

The AST profile uses the inherited exception-based opener and fused initialization. It measures 639 normalized bytes, 267 tokens, and 298 AST nodes, with a 360-byte gzip-9 stream. Nonempty malformed strings reject through Python's finite recursion limit; an ideal unbounded interpreter could diverge on those malformed inputs. This qualification is retained, and the profile is not the recommended total-input parser. An immutable-context continuation loop gives 296 AST nodes at the cost of 689 normalized bytes.

All measurements include input parsing calls, scope traversal, logical and arithmetic verification, and the acceptance driver. The shared Python interpreter and JSON library remain outside the source counts, as in the inspected drafts. The measured primary program imports only `json`. Ordinary nonzero process exit rejects; acceptance requires exit zero and the exact line `ACCEPT`.

The delivered finite suite regenerates the canonical proof examples, compares the producer against its documented frozen-source adaptation, checks the entire public programs normally and with Python optimization, and checks malformed-input and guard-removal controls. The false statement controls include $0 = 1$, $I(1, 1; v.v) = 0$, and $I(0, 1; v.0) = 0$. Separate controls test a forbidden compound-induction step and a free parameter in a requested closed goal. These controls are evidence about the implementation; the universal correctness and bi-interpretability claims rest on the written arguments and inherited bridge. No Lean or Rocq formalization is claimed.

3.6 The entire selected executable

```
import json
k = 0
i = lambda a, t=None, d=0: [i(b, t, d + 5 - len(a)) for b in a] if isinstance(a, list) else a if k <=
     a < d else d % 2 and i(t, d=3)

def c(a, g):
    global k
    if i(a) in g:
        return
    m, s, u, t = q()
    if m:
        c(m, g)
        if m in ([s, u], not s and [t] * 3) and i(t, u) == a:
            a = i(t, s)
        h = m * 3
    elif m is None:
        h = [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]
    else:
        h = a = i(*a, t, 2)
    k -= 1
    c(i(a, [k]), g + [i(h, k if m or len(a) % 2 == m else 0)])
q = json.load(open(0)).pop
c(q(), [])
print('ACCEPT')
```

4 Conjunctive induction still leaves a symmetry obstruction

The earlier eventually-constant-power obstruction in the inspected `ipra_research_2026-11-10` draft, specifically `theory/finite_parameter_obstruction.tex`, includes equation induction, finite conjunctions of equations, and universal Horn arithmetic laws [4]. That source already attributes the finite-parameter involution argument to earlier unary-amalgam work; neither the power construction nor the automorphism method is claimed here as new. The result below first extends that particular construction to existential witnesses: both the induction matrices and the regular axioms may contain positive existential quantification. A further closure theorem allows unrestricted finite alternation of existential and universal quantifiers, provided that the formulas use conjunction and no disjunction, implication, or negation internally. Both layers extend the existing construction; neither is presented as a literature-wide priority claim or a positive axiomatization result.

4.1 Exact hypotheses

Let $\mathcal{N}$ be any one-sorted finitary expansion of the standard structure $(\mathbb{N}, 0, S)$ by total functions and relations. The signature can be infinite; each symbol has finite arity, and each formula is finite. In particular, it can contain every primitive-recursive function. A *primitive-positive* formula is built from atomic formulas and $\top$ by finite conjunction and existential quantification. Neither disjunction, implication, negation, nor universal quantification is permitted inside such a formula.

We permit two kinds of regular axioms:

$$\forall \vec{x}\,(\varphi(\vec{x}) \to \psi(\vec{x})), \qquad \forall \vec{x}\,(\varphi(\vec{x}) \to \bot), \tag{Reg}$$

where φ, ψ are primitive-positive and the displayed sentence is true in $\mathcal{N}$. The second form explicitly includes negative constraints such as $0 = S0 \to \bot$. Terminology differs on whether regular logic includes falsity, so this form is listed separately.

For every primitive-positive $A(n, \vec{p})$, include the full induction sentence

$$\forall \vec{p}\,[(A(0, \vec{p}) \wedge \forall n\,(A(n, \vec{p}) \to A(Sn, \vec{p}))) \to \forall n\,A(n, \vec{p})]\,. \tag{PPI}$$

Parameters $\vec{p}$ remain fixed during induction. This is equivalent to the usual contextual induction rule for this class of matrices in intuitionistic first-order logic. Retained assumptions can contain arbitrary formulas. The claim below does not restrict them to primitive-positive formulas.

Let $T_{\mathrm{pp}}(\mathcal{N})$ be intuitionistic first-order logic with equality, all sentences (Reg), and all instances (PPI). This semantic upper theory need not have a recursive axiom set. The obstruction also applies to any recursively axiomatized subtheory containing the same symbols and using only axioms from these classes. More generally, it applies to any theory satisfied by the model constructed below.

4.2 The model and its positive existential formulas

Write

$$E = \{a \colon \mathbb{N} \to \mathbb{N} : a \text{ is eventually constant}\}.$$

Interpret every function of $\mathcal{N}$ coordinatewise. For an r-ary relation R, put

$$E \models R(a_1, \dots, a_r) \quad \Longleftrightarrow \quad \mathcal{N} \models R(a_1(i), \dots, a_r(i)) \text{ for every } i \in \mathbb{N}.$$

Equality is actual equality of sequences, and constants are constant sequences. Finitarity ensures that coordinatewise functions take eventually constant inputs to eventually constant outputs. Denote the resulting structure by $\mathcal{E}$.

Lemma 4.1 (Primitive-positive transfer)**.** *For every primitive-positive $\varphi(\vec{x})$ and every finite tuple $\vec{a}$ from E,*

$$\mathcal{E} \models \varphi(\vec{a}) \quad \Longleftrightarrow \quad \mathcal{N} \models \varphi(\vec{a}(i)) \text{ for every } i \in \mathbb{N}. \tag{PP}$$

Proof. Use external induction on the fixed finite formula. Atomic formulas and finite conjunctions follow immediately from the coordinatewise definitions. The forward existential direction follows by evaluating a witness at each coordinate.

For the reverse existential direction, suppose $\mathcal{N} \models \exists y\, \theta(y, \vec{a}(i))$ at every coordinate, and assume the transfer statement for θ. There is a finite set F such that $\vec{a}(i) = \vec{a}_\infty$ for every $i \notin F$. Choose a witness b_∞ for the common tail tuple, and a witness b_i separately for each of the finitely many coordinates $i \in F$. The sequence with values b_i on F and b_∞ elsewhere belongs to E. The induction hypothesis gives $\mathcal{E} \models \theta(b, \vec{a})$, hence the required existential statement. Only finitely many witness choices are used. Nested or tuple existential quantifiers are handled by repeated use of this argument. □

Lemma 4.2 (Regular axioms and primitive-positive induction)**.** *The classical structure $\mathcal{E}$ satisfies every axiom of $T_{\mathrm{pp}}(\mathcal{N})$.*

Proof. For a regular axiom, a true antecedent in $\mathcal{E}$ gives a true antecedent at every coordinate by Lemma 4.1. Validity in $\mathcal{N}$ gives the conclusion at every coordinate; the reverse direction of the lemma reconstructs its witnesses in $\mathcal{E}$. For a negative regular constraint, a true antecedent in $\mathcal{E}$ would give a true antecedent in $\mathcal{N}$ at coordinate zero, contrary to the constraint's validity there.

Fix an instance of (PPI) and a parameter tuple $\vec{p} \in E$. Suppose its base and progress hypotheses hold in $\mathcal{E}$. Fix a coordinate i. The transfer lemma gives $\mathcal{N} \models A(0, \vec{p}(i))$. If $\mathcal{N} \models A(k, \vec{p}(i))$, form the sequence a having value k at coordinate i and value 0 elsewhere. At coordinate i, $A(a(i), \vec{p}(i))$ holds by the supposition; at every other coordinate it holds by the transferred base hypothesis. The reverse transfer lemma therefore gives $\mathcal{E} \models A(a, \vec{p})$. Apply the assumed progress sentence in $\mathcal{E}$, and then transfer back at coordinate i. The result is $\mathcal{N} \models A(k+1, \vec{p}(i))$.

Ordinary external induction on the standard number k now gives $\mathcal{N} \models A(k, \vec{p}(i))$ for every k. This holds at each coordinate i. For any $a \in E$, the reverse transfer lemma gives $\mathcal{E} \models A(a, \vec{p})$. Hence the whole induction instance holds. The parameter tuple has remained fixed throughout. Nothing here assumes that witnesses for A are terms or are uniformly definable functions of n. □

A classical model is a one-node intuitionistic Kripke model. Thus the lemma supplies a model of the intuitionistic theory, not merely evidence about its classical restriction. In particular, any derivation using arbitrarily quantified retained assumptions must be valid in this model.

For orientation, successor coverage already fails on the original domain. The sequence equal to 1 at one coordinate and 0 elsewhere is not the zero sequence and is not a successor, since every successor sequence has a positive value at every coordinate. This observation alone would not exclude an interpretation on a different definable domain; the following symmetry argument is what handles that freedom.

4.3 The automorphism obstruction

Lemma 4.3 (Finite parameters leave an involution)**.** *For every finite tuple $\vec{p} \in E$, the structure $(\mathcal{E}, \vec{p})$ has a nonidentity automorphism of order two.*

Proof. Outside a finite set, every component of $\vec{p}$ is constant. Choose two distinct coordinates i, j outside this set and swap them. Coordinatewise interpretation makes the swap an automorphism

preserving all function and relation symbols, irrespective of the size of the signature. It fixes $\vec{p}$ and has order two. It is nonidentity because the sequence equal to 1 at i and 0 elsewhere is moved to the corresponding sequence supported at j. □

Theorem 4.4 (No ordinary bi-interpretation with iPRA)**.** *$T_{\mathrm{pp}}(\mathcal{N})$ is not bi-interpretable with iPRA by ordinary finitary first-order syntactic interpretations, of finite dimension, whose composites are provably definably isomorphic to the respective identity interpretations. Arbitrary formula-defined domains, relation/function graphs, and quotient equality are allowed. Each model may use a fixed finite parameter tuple, including parameters for the return isomorphisms, as long as the claimed interpretations and isomorphisms are required to exist in that model. In particular, this covers a syntactic parameter scheme whose finite parameter condition is provably inhabited and under which all these facts are proved uniformly. The same conclusion holds for every theory having $\mathcal{E}$ as a model. A parameter condition that simply excludes $\mathcal{E}$ and is not proved inhabited there is outside the claim.*

Proof. Every classical model of iPRA has a definable linear order. For example, in the standard primitive-recursive language use $x \leq y \leftrightarrow \exists z\,(x + z = y)$; iPRA proves that this is a linear order. An order-preserving automorphism of finite order is the identity: if $a < \sigma(a)$, iterating the strict inequality gives $a < \sigma(a) < \cdots < \sigma^r(a) = a$, and the case $\sigma(a) < a$ is analogous.

Suppose there were a bi-interpretation. Apply the interpretation to $\mathcal{E}$ to obtain a classical iPRA model $\mathcal{M}$. Collect the finitely many parameters of both interpretations and their definable return isomorphisms; representatives of any parameters in an interpreted quotient add only finitely many elements of E. Lemma 4.3 supplies a nonidentity involution σ fixing all of them.

Every first-order interpretation carries such an automorphism to the interpreted structure, including when its domain is a definable set of tuples modulo a definable equivalence relation. The induced automorphism $\widehat{\sigma}$ of $\mathcal{M}$ has order dividing two, so the preceding linear-order argument makes it the identity. Its further interpretation is therefore the identity on the double-interpreted structure. The source-side return isomorphism is definable with parameters fixed by σ, hence is equivariant:

$$\eta(\sigma(a)) = \widehat{\widehat{\sigma}}(\eta(a)) = \eta(a).$$

Since η is injective, $\sigma(a) = a$ for every $a \in E$, contrary to the choice of σ. □

Corollary 4.5 (Additional sorts do not evade this obstruction)**.** *Let $\mathcal{A}$ be a finitary many-sorted structure with nonempty sorts and a distinguished numerical sort that is the standard $(\mathbb{N}, 0, S)$. Other sorts may, for example, consist of finitary objects or specified numerical functions, with arbitrary total finitary operations and relations between the sorts. Form the intuitionistic theory of all true many-sorted regular sequents and negative regular constraints, together with induction on the numerical sort for every primitive-positive matrix. It is not bi-interpretable with iPRA by the ordinary syntactic interpretations of Theorem 4.4. Interpretation domains may use finite products of any of the source sorts and definable quotients.*

Proof. For each sort with domain A_s, take the eventually constant sequences $\mathbb{N} \to A_s$, and interpret all symbols coordinatewise, using the same coordinate index set for every sort. A fixed formula and its parameter tuple involve only finitely many sorts and coordinates with exceptional values. The primitive-positive transfer proof therefore works unchanged: existential witnesses have their specified sorts, and finite exceptional witnesses plus one common tail witness give an eventually constant witness of that sort. This proves every true regular axiom.

For induction, the varying sequence $a_{i,k}$ belongs to the distinguished numerical sort. All other parameter sorts stay fixed. The proof of Lemma 4.2 consequently proves every primitive-positive numerical-induction instance, even when witnesses and parameters range over other sorts.

Swap two coordinates outside the combined finite support of all interpretation and return-map parameters, simultaneously in every sort. This is an automorphism of the whole many-sorted structure. It is nonidentity already on the numerical sort, by the sequence supported at one of the swapped coordinates. Its induced automorphism on an interpreted iPRA model must be the identity. Definability of the source return isomorphism, on every source sort, then makes the original automorphism the identity on every sort. The numerical-sort contradiction suffices. No faithfulness of the action on each other sort is assumed or needed. □

4.4 Universal quantifiers give a stronger closure theorem

The primitive-positive result is not the limit of this method. Let $\mathcal{C}$ be the class of *conjunctive formulas* generated from atomic formulas and $\top$ by $\wedge$, $\exists$, and $\forall$. Quantifier alternation is unrestricted. There is still no disjunction, implication, or negation inside a matrix from $\mathcal{C}$. The explicit grammar is the convention used here; no identification with a differently defined syntactic fragment is assumed.

Theorem 4.6 (Conjunctive transfer and induction)**.** *For every $\varphi(\vec{x}) \in \mathcal{C}$,*

$$\mathcal{E} \models \varphi(\vec{a}) \quad \Longleftrightarrow \quad \mathcal{N} \models \varphi(\vec{a}(i)) \text{ for every } i \in \mathbb{N}.$$

Consequently $\mathcal{E}$ satisfies induction for every matrix in $\mathcal{C}$, as well as every sentence true in $\mathcal{N}$ of the forms

$$\forall \vec{x}\,(\varphi(\vec{x}) \to \psi(\vec{x})), \qquad \forall \vec{x}\,(\varphi(\vec{x}) \to \bot), \qquad \varphi, \psi \in \mathcal{C}.$$

These conclusions hold also in the many-sorted setting of Corollary 4.5, with induction in the numerical sort. The resulting stronger theories are still not bi-interpretable with iPRA under the interpretations specified in Theorem 4.4.

Proof. Induct on the fixed finite formula. The atomic and conjunction cases are unchanged. The existential proof of Lemma 4.1 uses only that the body already satisfies transfer and that its finite parameter tuple has a common tail. It therefore works even when the body contains universal quantifiers.

For the universal case, suppose $\mathcal{E} \models \forall y\, \theta(y, \vec{a})$. Fix a coordinate i and any element b of the corresponding standard sort. The constant sequence $\overline{b}$ is an element of the sequence sort. Instantiate the universal statement at $\overline{b}$ and apply the induction hypothesis at coordinate i. This yields $\mathcal{N} \models \theta(b, \vec{a}(i))$. As b was arbitrary, the coordinate satisfies the universal statement. Conversely, if every coordinate satisfies that universal statement, then for every sequence b every coordinate satisfies $\theta(b(i), \vec{a}(i))$. The induction hypothesis gives $\mathcal{E} \models \theta(b, \vec{a})$ for every such b, which is the required universal statement in $\mathcal{E}$.

The proofs of regular-axiom preservation and induction in Lemma 4.2 use only exact coordinatewise transfer; apply them to this larger class. The finite-parameter involution and return-isomorphism obstruction use only the unchanged underlying model, so they apply verbatim. The proof allows arbitrarily alternating quantifiers; it supplies no term-witness or decidability assumption. □

The one-sorted primitive-positive and many-sorted statements above are thus special cases. Even unbounded quantifier alternation in this conjunctive fragment does not force the uniform branch information missing from successor coverage.

Function objects and extensionality. The many-sorted statement also applies when a sort consists of specified actual numerical functions and the signature includes a total finitary application operation. Extensionality has the allowed form

$$(\forall n\ \ \mathrm{app}(f, n) = \mathrm{app}(g, n)) \to f = g.$$

Its antecedent is conjunctive, so the sequence model preserves it. Thus adding function objects, application, and extensionality does not by itself evade the obstruction. Equational combinator laws and any other standard-true conjunctive sequents may be included as well. This observation does not address a calculus with additional principles outside the stated class; it identifies the exact assumptions under which that possible change of domain still retains the symmetry.

Scope. The theorem is about ordinary first-order bi-interpretations, not equiconsistency alone. It allows existentially specified witnesses and conjunctive induction of arbitrary quantifier alternation, so changing a term-defined operation to a positive existential graph does not by itself evade this obstruction. It does not rule out arbitrary one-way interpretations, nonregular logical rules, disjunctive coverage axioms, or arbitrary formula-defined expansions that are not interpreted pointwise. Nor does it yield a numerical lower bound for proof-checker source size. Its practical use is to rule out an entire family of proposed positive existential completions before attempting a full bi-interpretation proof.

5 Executable evidence, attribution, and research limits

5.1 What the delivered checks establish

The selected kernel and its four explanatory or ancillary counterparts were checked on 31 named proof and adversarial fixtures, under ordinary Python and Python's `-O` optimization mode. The resulting 310 complete-program executions matched their expected outcomes. Forty malformed-input checks and 24 targeted guard-control executions also passed. Successful checking requires both process exit zero and exactly the line `ACCEPT`; a traceback alone is not a verdict convention.

The positive examples include zero computation with an arbitrary seed, successor computation using the previous state, retained parameters, transport, closure specialization, paired introductions and eliminations, zero witnesses, and contextual induction. The fixture producer is untrusted and is not imported by a checker. It is independently regenerated from two frozen upstream producers, and the generated packets are compared with the delivered fixtures before checking.

The negative controls target actual failure modes. Removing selected guards makes deliberately changed kernels accept false equations such as $0 = 1$, $I(1, 1; v.v) = 0$, or $I(0, 1; v.0) = 0$. Other controls exercise an attempted induction on a compound formula and an unallocated parameter in a requested closed goal. These are targeted checks of the trust boundary. They are not a proof that no other implementation defect exists, and they are not a formal verification of the metatheorems.

The ordinary-list primary traversal handles finite JSON trees by the specified checking operations, subject to physical resource bounds. The ancillary exception-based opener has the additional qualification that rejection of certain malformed nonempty strings uses the runtime's recursion limit. Large valid proofs can exceed ordinary memory or stack limits in every delivered implementation. The mathematical completeness statement is about unbounded finite derivations, not an assertion that a fixed physical process accepts every arbitrarily large proof.

5.2 Exact examples for the symmetry theorem

The file `code/eventually_constant.py` represents an eventually constant sequence by its constant tail and a finite exception map. Its output illustrates the infinite model exactly; it does not replace the transfer and induction proofs by bounded experiments. Let e_i be one at coordinate i and zero elsewhere. Then:

Property	In the sequence model	Reason
$e_i = 0$	False	Coordinate i is one
$\exists y\ (e_i = Sy)$	False	All other coordinates are zero
$\mathrm{sg}(e_i) = 0$	False	Its ith coordinate is one
$\mathrm{sg}(e_i) = 1$	False	Its tail is zero
$e_i \leq e_j$, $i \neq j$	False	Fails at coordinate i
$e_j \leq e_i$, $i \neq j$	False	Fails at coordinate j

Here $\mathrm{sg}(0) = 0$ and $\mathrm{sg}(Sn) = 1$, interpreted coordinatewise. All its true conjunctive identities still hold. The failure is uniform disjunctive information: being Boolean at every coordinate does not mean being globally the constant zero or the constant one. This explains why standard Boolean coding cannot silently turn the model's conjunctive induction into full quantifier-free induction with the usual logical equivalences.

The example also swaps two coordinates outside the finite supports of specified parameters, fixes those parameters, and moves e_i to e_j. The theorem proves this for every finite tuple and every finitary pointwise signature. The program merely makes representative instances easy to inspect.

5.3 What was investigated but not claimed

Two broader positive directions were examined. First, replacing computation supply with numerical definitional conversion would remove an explicit branch, but a correct reducer must respect the codec's dummy slots, state binder, neutral counters, and rigid parameters. Its extra machinery did not yield a smaller implemented checker. Moreover, a uniformly total primitive-recursive evaluator for arbitrary primitive-recursive descriptions is impossible by the usual diagonal argument. A recursive normalizer or a bounded conversion certificate is a different option and is not excluded by that observation.

Second, genuine binders in the proof-certificate grammar could let the opener allocate unforgeable eigenparameters, replacing predictable global negative labels. The flat instruction stream currently needs those labels so that later instructions can refer to earlier eigenparameters. A new scoped grammar would therefore require a corresponding proof-opening discipline. No complete smaller construction was obtained along that route.

Deleting DI-case for literal equations is not justified. An ordinary conditional closure can hold because its zero antecedent is false, without its matrix being progressive at every argument. Reinterpreting atomic paired formulas as progressiveness also requires a matching elimination rule and a proof of the source-side formula composite. These obligations cannot be discharged just by recovering some ordinary logical macros. The existing drafts already analyze this difficulty; it is not claimed as a new negative result here.

5.4 Reproduction and the remaining target

From a fresh extraction, with Python 3.12 and a normal PDFLaTeX installation:

```
python3 reproduce.py
python3 code/checker.py < tests/fixtures/I0_arbitrary_seed.json
python3 code/checker.py < tests/fixtures/contextual_induction_seed_congruence.json
python3 build.py
```

The first command verifies payload hashes, recomputes the complete-module and gzip measurements, checks producer provenance, runs the delivered proof fixtures and guard controls, and reproduces the

exact sequence examples. It needs no network. The last command rebuilds the report without shell escape. A standalone LaTeX source is supplied as well as the modular sources.

The package preserves frozen comparison programs and the exhaustive source inventory. Repeating the complete corpus scan additionally needs the thirteen original archives from the pinned GitHub revision; `reference/census.py` records the scan procedure. The regular reproduction command uses the included frozen programs and requires none of the original ZIP downloads.

The outcome is intentionally narrower than an absolute solution to the research problem. There is a complete implementation below the supplied byte and token envelope, and a newly extended obstruction that rules out an otherwise tempting family of changed-language and changed-domain approaches. A significant new positive axiom basis, a checker below all three structural minima in one program, and a global simplicity theorem remain unproved.

Attribution and the complete inherited bridge

The next five sections are reproduced unchanged from the `ipra_uniform_predecessor_2026-11-10.zip` archive at the pinned commit [1, 2]. They contain the logical reconstruction, the ordinary first-order comparison theory, the scalar predecessor, actual equality reflection, and the primitive-recursion compiler. They are included to make the asserted iPRA equivalence inspectable in this package. These results are inherited and are not claimed as new here. Their exact hashes and original member paths are recorded in `evidence/inherited_provenance.json`. The repository's MIT No Attribution license is included in `reference/PROVEIT_LICENSE.txt`.

The clause notation in the first appendix is the older presentation of the same conditional closure used as Q in the main text. A clause $\langle B(0); y.B(y)\rangle$ is represented by $\mathsf{Q}_y B$; an ordinary assumption F is a constant clause $\langle H; v.F\rangle$, with H a fixed computation theorem, and corresponds to the formula assumption F, not to the tautological closure $\mathsf{Q}_v F$. Conversely $\mathsf{Q}_y B$ is interderivable with $\mathsf{D}_{x,y}B$ for an unused x. The public-Q presentation in the main text and the appendix's clause presentation therefore have the same closed native theorems. This interface is also used by the frozen upstream checkers; it is not an extra logical oracle in the new program.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \quad \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x,y) \qquad \Gamma, \langle B(a,0); y.B(a,y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y}E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \le 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D *are derivable without using the progress choice.*

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z} M_z(A, B), \\ A \wedge_* B &:= \mathsf{D}_{u,z} G_z M_{p(z)}(A, B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &:= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z), \\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B \rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x(B(x,0) \Rightarrow_* \forall_* y\, B(x,y)). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a,0)$ that clause supplies $B(a,y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y B^\circ(x,y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

B.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem B.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

C Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [10] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n,a;u.h), \qquad I(0,a;u.h) = a, \qquad I(Sn,a;u.h) = h[I(n,a;u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

C.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n,a,b) = I(n,a;u.b), \qquad Z(n) = C(n,1,0), \qquad a+b = I(b,a;u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0,a,b) = a, \qquad C(Sn,a,b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{3}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{4}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

C.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{5}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{6}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{7}$$

For completeness, its successor calculation is

$$G(3Sn) = G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4,$$
$$3G(Sn) + N(Sn) = 3G(n) + 3N(n) + 3 + Z(N(n)).$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (7) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{8}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (7); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

C.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{9}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (8), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{10}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{11}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

C.4 Saturated iteration gives predecessor

We now need only the three equations (11). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (12) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (13) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (14)$$

The base is (13). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (15)$$

The first uses the comparator recurrence. The second uses (14) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (16)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{17}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (16) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

C.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

D From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [11]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix C, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

D.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion.

In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{18}$$
$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{19}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x \, (x = 0 \vee x = S(Px)), \tag{20}$$
$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{21}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p} \, \left[(E(0, \vec{p}) \wedge \forall n (E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n \, E(n, \vec{p}) \right]. \tag{22}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (22) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

D.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{23}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{24}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{25}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (18). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (24); symmetry and (21) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{26}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{27}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{28}$$

To prove the range assertion, split (20). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (19) also give, for all u, v,

$$C(u, v, v) = v. \tag{29}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{30}$$

If $u = 1 = S0$, (25) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

D.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{31}$$

$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{32}$$

Using these equations and (22), we now derive

$$0 + y = y, \tag{33}$$

$$(Sx) + y = S(x + y), \tag{34}$$

$$d(Sx, Sy) = d(x, y), \tag{35}$$

$$d(Sa, a) = 1. \tag{36}$$

Each induction below has a single equation as its induction formula.

For (33), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (34), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (34) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{37}$$

Indeed, both sides are $S(y + r)$.

For (35), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (36). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (35) and the induction hypothesis.

D.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)) = x. \tag{38}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (33) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (29).

For the step, assume (38) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (38) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (37) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (38). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{39}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (38). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (36) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{40}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{41}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{42}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (27), which is equivalent to $d(Sa, b) = 1$ by (30), and then to $a = b$ by (39). The range assertion is (26) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (41) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{43}$$

is a theorem, not an extra hypothesis.

D.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s, t), \qquad b_\bot = 1, \tag{44}$$

$$b_{A \wedge D} = C(b_A, b_D, 1), \tag{45}$$

$$b_{A \vee D} = C(b_A, 0, b_D), \tag{46}$$

$$b_{A \to D} = C(b_A, b_D, 0). \tag{47}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{48}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (41)–(42). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (48) for every quantifier-free formula.

D.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (48), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

E Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [12]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

E.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a,b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a,b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a,b)) = a + b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

E.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u,\vec{p}) = \pi(SL(u), h^*(L(u), H(u),\vec{p})),$$

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u,\vec{p})), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n,\vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

E.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

Reference proof for the original arity codec

The following section is reproduced unchanged from the arity-calculus archive [3]. It concerns that archive's original computation packet and original verdict driver. The exact conversion in Section 3, rather than an identification of different programs, transfers its soundness argument to the selected checker. The reference's statements about its own measured profiles are not the measurements of the present program.

F A uniform arity encoding and its checking theorem

The arithmetic and object logic remain the scalar-iteration Eq/D calculus specified above. The change is in its finite certificate representation. A paired formula is unary, an equation is binary, and an internal closure is ternary:

$$\mathsf{D}_{x,y}B \longleftrightarrow [B], \qquad s = t \longleftrightarrow [s,t], \qquad \mathsf{Q}_y B \longleftrightarrow [B,B,B]. \tag{Arity encoding}$$

The three copies in a closure are representation redundancy: only the third has mathematical content. Their benefit is that tripling a unary major constructs exactly its closure. Tripling any other nonempty formula array produces an array of arity at least six, which will denote truth. This makes the other major cases neutral without a separate test.

The previous generated-assumption and constructed-computation interfaces are inherited from the corpus [6, 7]. The new encoding makes a single arithmetic expression, $5 - \text{arity}$, perform the necessary depth adjustments. Its soundness depends on an explicit interpretation of surplus syntax. It would not be sound to drop arity checks while continuing to treat every surplus array as an unconstrained new connective.

F.1 The complete sorted interpretation of arrays

Write $\top$ for the fixed equation $0 = 0$. A protected numerical slot has width two. The numerical root has depth three: the coordinates $0, 1, 2$ all denote zero, and under p protected slots the depth is $3 + 2p$. Coordinates $3 + 2j$ and $4 + 2j$ name slot j, counted from the innermost slot, for $0 \le j < p$. Formula depth is $2p$ under the same p slots. Negative coordinates admitted by the current global floor name rigid numerical parameters.

The decoder is sorted by the position in which a tree is used. Let ρ be its ordered environment of protected numerical slots. The notation (a, ρ) means prepend a slot of value a. A superscript $\sharp$ denotes a child decoded in the indicated environment.

Numerical arity	Meaning	Relevant child environment
0	0	No child
1: $[b]$	$Sb^\sharp$	$(0, 0, \rho)$
2	0	Annotations ignored
3: $[n, a, h]$	$I(n^\sharp, a^\sharp; v.h^\sharp)$	$(0, \rho)$ for n, a; (v, ρ) for h
≥ 4	0	All children ignored

Formula arity	Meaning	Relevant child environment
0	$\top$	No child
1: $[B]$	$\mathsf{D}_{x,y}B^\sharp$	(y, x, ρ)
2: $[s, t]$	$s^\sharp = t^\sharp$	ρ numerically
3: $[P, U, B]$	$\mathsf{Q}_y B^\sharp$	(y, ρ) for B
≥ 4	$\top$	All children ignored

The first two entries of a ternary formula are annotations. A canonical closure copies its body into both of them, as in (Arity encoding). Successfully admitted scalar formula padding also denotes $\top$. An internal closure has the ordinary meaning $B(0) \to \forall y B(y)$; it is not an equation and is not eligible for compound induction.

Canonical encoded object expressions use arrays and integer coordinates, respect these slots, and contain no high-arity node. The checker itself still generates high-arity truth entries in administrative contexts, even during a canonical proof. In particular, constructing a successor beneath positive bound coordinates requires passing through two dummy zero slots. Constructing an iterator requires one new slot: zero in its counter and seed, and its genuine state variable in the update. Rigid negative parameter names are unaffected by these shifts.

The additional raw shapes are certificate aliases with the displayed meanings. They are not independent predicate or function symbols added to the object theory. Our exact completeness statement concerns the canonical native language. We do not require every possible raw spelling of a semantic alias to be interchangeable under the same raw certificate.

F.2 One depth calculation replaces the lookup table

For every array the opener recursively adds $5 - \text{arity}$ to its depth. The meaningful cases are

arity	1	2	3
depth increment	4	3	2.

Unary D adds two numerical binders; unary successor adds two dummy zero slots. Binary equality changes formula depth $2p$ to numerical depth $3 + 2p$. Ternary Q adds one numerical binder; ternary iteration adds one state slot, or one dummy slot in its counter and initializer. The same calculation therefore covers all relevant constructors.

Binary numerical annotations denote zero, so their parity-changing traversal has no mathematical effect. Empty arrays have no children. At arity at least four, the entire node already denotes a constant. Even a zero or negative depth increment below such a node cannot expose one of its ignored children as a meaningful variable. Traversal may still reject an alias; it cannot change its stipulated interpretation.

This is a semantic assertion, not raw idempotence of the opener. For example, at floor -1, opening a six-element array of zeros with image -1 can yield six copies of -1, while a later no-image opening of that result fails at a negative child depth. Both formula arrays denote $\top$. The proof below only concerns successful calls at meaningful sorted positions. Canonical input object expressions contain no surplus arrays; the administrative exports that generate them are justified separately as truth entries.

F.3 Opening and scope

Write $i(a, t, d)$ for the executable opener at floor k. It preserves a scalar when $k \leq a < d$. At a numerical position, every other relevant scalar is the distinguished substitution hole; the supplied

image is checked at numerical root depth three before insertion. At a formula position scalar padding is truth and carries no numerical hole. Missing images at used numerical holes reject.

Lemma F.1 (Semantic opening)**.** *For every successful call at a meaningful formula root of depth $2p$ or numerical root of depth $3 + 2p$, with $p \geq 0$, decoding its result is capture-free filling of the distinguished hole in the decoded input template. These depths protect p numerical slots; admitted rigid parameters are preserved. If an image is semantically unused, zero may be chosen as its mathematical representative. Recursive calls at negative depths inside ignored annotations need no independent sorted interpretation.*

Proof. Induct on the finite raw tree. At a relevant scalar, the interval comparison is exactly the protected-coordinate test. At a meaningful unary, binary, or ternary node, the preceding tables supply precisely the required additional slots. At numerical binary annotations, high-arity nodes, and ignored closure fields, the relevant mathematical meaning is constant or absent, so the conclusion is immediate regardless of traversal inside the annotation.

An inserted image is checked at its own numerical root. Its free parameters are rigid negative names, and it has no unbound positive coordinate there. Insertion beneath protected slots therefore captures none of its parameters; its internal binders retain their relative indices. This proves the inductive step for substitution as well as for mere traversal. □

A successful no-image check consequently preserves the interpretation of its checked closed expression. Raw structural equality of checked outputs implies equality of their decoded meanings. The checker does not infer equality merely from a semantic comparison of two different programs or numerical terms.

The global floor decreases by one at each non-hypothesis continuation. If its old value is K, the integer $K - 1$ is the designated fresh eigenparameter. With integer coordinates it is the only new name. If fractional coordinates are accepted, other names in $[K - 1, K)$ may also occur in the finite certificate. These are additional fresh parameters. They must not be silently equated to the designated integer eigenparameter.

Previously checked relevant goals and retained hypotheses mention none of these new names: their earlier no-image checks would reject a relevant numerical occurrence below the old floor. Coordinates inside ignored annotations do not contribute free variables to the decoded formula. This is the freshness invariant used in every rule below.

F.4 A computation hypothesis only needs a zero instance

The shared continuation can open a generated arithmetic equation at a fresh parameter. That equation need not be a theorem uniformly in the new parameter. The following admissible rule is sufficient.

Lemma F.2 (Supply with a known zero instance)**.** *Suppose $\vec{a}$ is fresh for Γ, F, the calculus proves $H(\vec{0})$, and $\Gamma, H(\vec{a}) \vdash F$. Then $\Gamma \vdash F$.*

Proof. Substitute zero for every member of $\vec{a}$ in the second derivation. Freshness leaves Γ, F unchanged. Cut in the derivation of $H(\vec{0})$. Both substitution and cut are admissible in the native calculus. This is a finite proof transformation and does not use a consistency assumption. □

All names newly admitted when the floor changes belong to $\vec{a}$; for noninteger aliases this includes the finite collection in $[K - 1, K)$ that actually occurs. On decoding, their simultaneous zero substitution agrees with opening the generated expression at zero using the old floor. Previously admitted

parameters are preserved. New rigid names and unprotected placeholders both become zero. The opener may keep different raw zero spellings, but their decoded values agree.

The constructed-computation packet supplies a triple $s = [n, b, r]$ and a mode u. In successor mode its generated equation has raw form

$$[\,[[n], b, r],\ \ i(r, s, 3)\,], \tag{Step supply}$$

while in zero mode it has form

$$[\,[u, b, r],\ \ b\,]. \tag{Zero supply}$$

Here successor mode means that u is truthy; the zero mode has falsy u. These expressions describe actual arrays, before their final opening. In (Step supply) the counter is constructed as a successor, rather than accepted on an unsupported assertion that it is one. This construction is inherited from the corresponding draft interface [7].

Lemma F.3 (Sound constructed computation)**.** *Every successful arithmetic supply has a decoded zero instance derivable from the two scalar-iteration computation schemas.*

Proof. First apply the simultaneous zero substitution described above. In zero mode every successful falsy numerical counter denotes zero, so (Zero supply) is an $I0$ instance. In the exact-list profile the relevant accepted aliases are an empty array, numeric zero, false, and numerically equal zero values. Null, strings, and dictionaries cannot enter a meaningful numerical position through an unchecked coercion.

In successor mode the left counter is an actual successor. If the update uses its state in a semantically relevant position, opening it invokes the no-image check of the recursive source s at numerical root three. This checks the entire source, including its update. Only coordinates three and four are aliases for its single state slot. A genuinely escaping coordinate in that update is revisited during the recursive-image check and rejects. Its counter and initializer are likewise closed with their prescribed dummy zero slot.

Inserting the extra dummy slots of the displayed successor does not alter such a checked predecessor: extra unbound slots have value zero, and actual free parameters are rigid. Lemma F.1 therefore identifies the right side of (Step supply) with the update applied to exactly the recursive predecessor value occurring on its left. This is the IS schema.

If the update has no semantically relevant state occurrence, its decoded value is constant in that variable. Its zero-normalized value is the same in the source update and on the right, independently of the unused recursive image; this is again an IS instance. Traversal inside an ignored annotation may still inspect the image and reject. That removes a certificate and does not invalidate the argument for a successful one.

Finally, successful three-field source extraction cannot hide another JSON operator. A scalar is not iterable. A three-character string, or a three-key JSON dictionary, necessarily puts a nonempty string in a relevant source field, which the inspected opener rejects. The successful source is therefore the stated array triple. □

The recursive-image check in the state-dependent case is essential. Merely opening a final displayed equation at zero would not establish that its two state interpretations agree. Combining Lemma F.3 with Lemma F.2 justifies both actual image choices of the common continuation, including the branch that retains a fresh-parameter hypothesis.

F.5 Audit of the logical continuations

Hypothesis closure. The initial opener check and membership comparison select the same decoded context formula. Additional aliases can make a spelling less convenient, but cannot identify an equation with a false denotation solely by semantic evaluation.

Equality transport. The major must match $[s, u]$, and opening the proposed formula template at u must match the target. The continuation checks its opening at s. Lemma F.1 gives ordinary transport along the decoded equality. The argument applies to compound formulas, internal clauses, and truth aliases as well as equations.

Closure specialization. The closure pattern is the triplicate $[t, t, t]$. The guard forces the old instance to have numerical image zero; the new instance may have an arbitrary numerical image. This is exactly $B(0) \to \forall y B(y)$. The zero guard is indispensable. Without it, a closure with false zero instance and true successor instances would turn a proved successor instance into falsity.

Paired elimination. The export is always three concatenated copies of the major. A unary major $[B]$ therefore exports $[B, B, B]$. All three copies are opened identically; the third copy has precisely the witness instance required by paired elimination. The newly allocated witness is fresh for the retained context and conclusion.

An equation major instead exports arity six; a closure major exports arity nine; a larger array exports still larger arity. These are all truth aliases. Thus failed elimination endpoint claims have only the effect of adjoining a theorem. A numeric scalar major, if accepted as formula padding, exports scalar padding after multiplication and also denotes truth. Unsupported types reject.

A ternary formula with unequal annotations still denotes the closure of its third child. It fails the triplicate specialization pattern unless the three fields match. This loses a possible raw spelling, not a mathematical soundness condition. Canonical compilation uses triplicate closures throughout.

Paired introduction. The call `i(*a,t,2)` uses the function's argument count to enforce the relevant outer arity. For a unary goal $a = [B]$, it is exactly `i(B,t,2)`: the universal slot is protected while the existential witness is opened. For arity at least two, there are too many arguments and the call rejects. An empty goal can instead make the legal two-argument call `i(t,2)`. Its original target was already $\top$, so accepting it cannot add a false theorem.

For the unary goal, the subsequent successor opening gives the intended premise $B(t, Sz)$. The local hypothesis is $B(t, 0)$ in case mode and $B(t, z)$ in progress mode. The freshness invariant supplies the usual eigenvariable conditions.

Induction remains restricted. Among meaningful nonconstant matrix constructors, only binary equality has even arity. Unary paired formulas and ternary closures have odd arity and receive case introduction. Empty or surplus arrays that reach the progress branch denote the fixed equation $0 = 0$, whose induction is valid. Scalar padding that cannot provide a length rejects. The broader certificate alphabet therefore adds no induction on a quantified matrix.

Additional fresh parameters. An accepted subderivation can contain newly admitted fractional names, but the older context and target do not depend on them. Replace these auxiliary names by zero when closing the rule, retaining its designated integer eigenvariable where that rule needs one.

The common opening of an already checked retained goal leaves its decoded meaning unchanged. This handles the actual global-floor convention without pretending that every admissible negative number is an integer.

F.6 Canonical completeness, including zero witnesses

Rename a finite native proof's eigenvariables to fresh negative integers. Encode its terms with the slot conventions of Section F.1; encode paired formulas as unary and closures as triplicate. Hypothesis leaves consume no instruction. Transport, closure specialization, and paired elimination have their displayed packets. Arithmetic computation leaves use (Zero supply) or (Step supply) and then close the target by hypothesis lookup.

One representational issue requires an actual proof transformation. The packet's truthy witness field selects introduction. A literal zero witness therefore cannot simply be inserted into that field. Define

$$J(t) = I(0, t; v.0).$$

The computation axiom proves $J(t) = t$, and $J(t)$ has a nonempty array representation even when its value is zero. To introduce with a falsy witness spelling t, introduce instead with $J(t)$: transport its retained assumption to the assumption using t, use the original premise derivation, and transport the resulting successor instance back to the instance using $J(t)$. Derived symmetry supplies the reverse transport. This is a finite transformation of the premise, and the paired conclusion itself is unchanged. If the existential coordinate is unused, any closed truthy witness works without a conversion.

The scalar coordinate 1 is also a semantic zero at numerical root three, but that observation alone is not a completeness proof. Closure specialization and case introduction insert literal raw zero, and distinct zero aliases need not be interchangeable under a fixed raw proof spelling. The computation-backed box gives the required derivation instead of assuming representational interchangeability.

The mode zero selects progress at an equation. A different falsy mode, such as the empty array, selects case. These are certificate control tags, not numerical assertions about the witness. All steps in the compilation are finite and effective.

Theorem F.4 (Conservativity and completeness of the arity checker)**.** *Every accepted certificate has its decoded conclusion provable in the ordinary realization of the native calculus. For every canonical native end sequent, the internal checking judgment and the native calculus have the same consequence relation, in the idealized unbounded integer-and-array implementation. In particular, the public empty-context driver recognizes exactly the canonical closed theorems. The translation in each direction is effective on finite derivations.*

Proof. For soundness, induct on the finite accepting call tree. Hypothesis closure uses Lemma F.1; the logical cases are the audited continuations above. Arithmetic supply uses Lemmas F.3 and F.2. Every discarded annotation is accounted for by its zero or truth interpretation, and every fresh parameter by the scope invariant. For completeness, compile a finite native derivation by the preceding construction, including the boxed-witness transformation. Its canonical input expressions require no surplus-arity aliases or fractional names; generated administrative surplus arrays have exactly the truth meaning already proved. This proves both implications without using a finite test suite as a substitute for induction on derivations. □

The unchanged native calculus is bi-interpretable with full first-order iPRA by the inherited arithmetic and logical bridge [6, 9]. In particular,

$$(\mathsf{D}_{x,y}B)^\circ = \exists x(B^\circ(x, 0) \to \forall y B^\circ(x, y)).$$

The whole numerical domain and actual equality are retained. Its internally proved numerical round trips and its formula-composite derivations therefore transfer through Theorem F.4. We have changed the checked representation, not replaced bi-interpretability by standard-model agreement or supplied a new proof of the scalar predecessor theorem.

F.7 JSON and finite-runtime qualifications

The primary profile uses an ordinary list-type test. The smaller byte profile instead tests multiplication by zero against an empty list. Among standard JSON runtime values the latter recognizes exactly lists: numeric values and booleans produce numeric zero, strings produce an empty string, and null and dictionaries raise rather than becoming arrays. This argument assumes ordinary decoded JSON values, not arbitrary Python objects with overloaded methods.

Finite nonnegative real coordinates have the same interval interpretation: $[0, 3)$ is zero, and $[3 + 2j, 5 + 2j)$ names slot j. Finite negative coordinates in the admitted range are additional rigid names. Equal Python numerical values have the same decoded role, including Boolean aliases and signed zero. Nonfinite values admitted by Python's permissive JSON reader cannot survive a relevant closed numerical check. They may be filled as template holes or ignored inside a constant annotation; neither makes them unchecked parameters.

The alternative opener based on generic iteration has a separate runtime qualification. Empty strings and empty dictionaries traverse as empty arrays; a dictionary whose sole key is empty traverses as a unary array with an empty child. Dictionary values are not visited. Nonempty strings can rely on CPython's recursion limit to reject, as also occurs in a current corpus comparison profile [8]. The idealized structural-termination claim for the exact-list profile must not be transferred to that generic profile unchanged.

All executable profiles have ordinary finite memory, stack, and input limits. These can reject a sufficiently large otherwise valid proof. The formal consequence relation is the unbounded finite-certificate calculus; the delivered program checks certificates within its runtime resources. The measurements charge the entire delivered checker, while the semantic proof and canonical producer are supplied separately.

References

[1] Vladimir Reshetnikov, repository maintainer. *ProveIt, current draft collection*, commit `fb2dbd2a33c6b132671a54b029d0bf3cd98c633d`. https://github.com/VladimirReshetnikov/ProveIt/tree/fb2dbd2a33c6b132671a54b029d0bf3cd98c633d/docs/drafts. All archive references below refer to this immutable collection.

[2] *Uniform Predecessor Certificates for Intuitionistic Primitive Recursive Arithmetic.* `ipra_uniform_predecessor_2026-11-10.zip`, in [1]. Source of the predecessor instruction, current comparison leaders, and attributed complete logical/arithmetic appendices.

[3] *An Arity Calculus for iPRA.* `ipra_arity_calculus_2026-11-10.zip`, in [1]. Source of the unary paired binder, triplicate closure, numerical root depth three, and $5 -$ arity traversal.

[4] *Intuitionistic primitive recursive arithmetic research report.* `ipra_research_2026-11-10.zip`, in [1], especially `theory/finite_parameter_obstruction.tex`, the eventually constant power and its finite-conjunction generalization. The conjunctive transfer theorem in this report extends that construction.

[5] *Affine Depths for a Small iPRA Checker.* `ipra_affine_arity_2026-11-10.zip`, in [1].

[6] Inherited generated-assumption interface, reproduced and attributed in the current arity-calculus and uniform-predecessor archives, in [1].

[7] *Constructing computation sources for iPRA.* `ipra_constructed_computation_2026-11-10.zip`, in [1].

[8] *A one-state recursor for iPRA.* `ipra_transient_seed_2026-11-10.zip`, in [1].

[9] The pinned draft corpus [1]; this bibliography key is retained for the unchanged arity-codec reference section.

[10] Inherited scalar-predecessor development, reproduced with attribution as `reference/scalar_predecessor.tex` in [2] and here.

[11] Inherited forward-left equality-reflection development, reproduced with attribution as `reference/equality_reflection.tex` in [2] and here.

[12] Inherited pairing and primitive-recursion compiler, reproduced with attribution as `reference/primitive_recursion.tex` in [2] and here.

[13] Jeremy Avigad. *Proof mining*, lecture slides, 2004, slide 13. https://www.andrew.cmu.edu/user/avigad/Talks/asl04.pdf. Used only for the standard background distinction between first-order classical, first-order intuitionistic, and quantifier-free presentations.

[14] Yiannis N. Moschovakis. *Notes on Set Theory and Logic*, Chapter 3, Section 3J, primitive recursive arithmetic. https://www.math.ucla.edu/~dam/220b.1.11w/220notes3.pdf. Used only as a background reference for staged primitive-recursive descriptions and quantifier-free induction.